COMPLETE GRAPH DSA

JAVA INTERVIEW ALGORITHMS HANDBOOK

Fundamentals → Standard Interview Patterns → Advanced Algorithms

Definitions • Intuition • When to use • Complexity • Java implementations • Interview traps

Language: Java 17+ (standard library only)

Audience: DSA practice, product-company interviews, competitive-programming foundations

How to study: First understand the definition and pattern, then trace the code by hand, then solve the listed practice patterns without looking at the implementation.

How to use this handbook

This is a consolidated, interview-oriented Graph DSA reference. It is not merely a code dump: each algorithm begins with its definition, purpose, core intuition, complexity, and implementation.

• Vertices (V): nodes; Edges (E): connections between nodes.

• Adjacency list: for each vertex, store its neighbours. Usually the best default for sparse graphs.

• Directed graph: edges have direction. Undirected graph: an edge works both ways.

• Weighted graph: every edge has a cost/weight. Unweighted graph: each edge is treated equally.

• Unless stated otherwise, vertices are numbered 0 to n−1.

• For an undirected graph, add both u→v and v→u. For a directed graph, add only u→v.

• Use long for path costs when edge weights or path sums may exceed the int range.

Java graph template

```
import java.util.*;

class Graph {
    int n;
    List<List<Integer>> adj;

    Graph(int n) {
        this.n = n;
        adj = new ArrayList<>();
        for (int i = 0; i < n; i++) adj.add(new ArrayList<>());
    }

    void addUndirectedEdge(int u, int v) {
        adj.get(u).add(v);
        adj.get(v).add(u);
    }

    void addDirectedEdge(int u, int v) {
        adj.get(u).add(v);
    }
}
```

Important: The algorithms below are standalone methods or classes. Put them inside a class, add imports shown in the code, and adapt input parsing to the platform (LeetCode, GFG, or custom input).

Part 1 — Graph foundations and traversal

1. Breadth-First Search (BFS)

Definition / Intro: BFS traverses a graph level by level. It visits all vertices at distance 0, then distance 1, then distance 2, and so on.

When to use: Unweighted shortest path, minimum number of edges, level order, connected components, multi-source spread.

Core idea: Use a queue. Mark a node visited when it is enqueued, not when it is removed; this prevents duplicate queue entries.

Complexity: O(V + E) time and O(V) extra space with an adjacency list.

```
static List<Integer> bfs(List<List<Integer>> adj, int start) {
    int n = adj.size();
    boolean[] seen = new boolean[n];
    Queue<Integer> q = new ArrayDeque<>();
    List<Integer> order = new ArrayList<>();

    seen[start] = true;
    q.offer(start);

    while (!q.isEmpty()) {
        int u = q.poll();
        order.add(u);
```

```
            for (int v : adj.get(u)) {
                if (!seen[v]) {
                    seen[v] = true;
                    q.offer(v);
                }
            }
        }
        return order;
    }
```

Interview notes: For a disconnected graph, run BFS from every still-unvisited vertex to cover all components. BFS gives shortest paths only when every edge has equal cost.

2. Depth-First Search (DFS) — recursive

Definition / Intro: DFS explores one route as deeply as possible before backtracking.

When to use: Connected components, cycle detection, topological sorting, islands, backtracking-style graph exploration.

Core idea: Mark the current vertex, process it, then recursively visit each unvisited neighbour.

Complexity: O(V + E) time and O(V) visited/recursion space.

```
    static void dfs(int u, List<List<Integer>> adj, boolean[] seen,
                    List<Integer> order) {
        seen[u] = true;
        order.add(u);

        for (int v : adj.get(u)) {
            if (!seen[v]) dfs(v, adj, seen, order);
        }
    }
```

Interview notes: A very deep graph can overflow Java's call stack. Use iterative DFS when depth may be large.

3. Iterative DFS

Definition / Intro: Iterative DFS performs depth-first traversal using an explicit stack instead of the call stack.

When to use: Large inputs where recursive DFS may cause StackOverflowError.

Core idea: Push the start vertex, pop one vertex, and push its unvisited neighbours.

Complexity: O(V + E) time and O(V) space.

```
    static List<Integer> iterativeDfs(List<List<Integer>> adj, int start) {
        boolean[] seen = new boolean[adj.size()];
        Deque<Integer> st = new ArrayDeque<>();
        List<Integer> order = new ArrayList<>();
        st.push(start);

        while (!st.isEmpty()) {
            int u = st.pop();
            if (seen[u]) continue;
            seen[u] = true;
            order.add(u);

            for (int i = adj.get(u).size() - 1; i >= 0; i--) {
                int v = adj.get(u).get(i);
                if (!seen[v]) st.push(v);
            }
        }
        return order;
    }
```

Interview notes: Marking when popped is valid if duplicate stack entries are skipped as above. Marking on push is another valid design, but it can produce a different traversal order.

4. Connected Components

Definition / Intro: A connected component in an undirected graph is a maximal set of vertices where every pair is connected by some path.

When to use: Count groups, provinces, network clusters, number of islands (with grid adaptation).

Core idea: Start DFS/BFS from each unvisited vertex. Every new start begins one new component.

Complexity: O(V + E) time and O(V) space.

```
static int countComponents(List<List<Integer>> adj) {
    int n = adj.size();
    boolean[] seen = new boolean[n];
    int count = 0;

    for (int i = 0; i < n; i++) {
        if (!seen[i]) {
            count++;
            dfs(i, adj, seen, new ArrayList<>());
        }
    }
    return count;
}
```

Interview notes: This exact connected-component meaning is for undirected graphs. Directed graphs require notions such as strongly connected components.

5. Multi-source BFS

Definition / Intro: Multi-source BFS is ordinary BFS started simultaneously from multiple source vertices.

When to use: Rotting Oranges, nearest zero, nearest hospital, fire spread, distance to nearest marked cell.

Core idea: Insert every source into the queue at distance 0 before starting the BFS.

Complexity: O(V + E) time and O(V) space.

```
static int[] multiSourceBfs(List<List<Integer>> adj, List<Integer> sources) {
    int n = adj.size();
    int[] dist = new int[n];
    Arrays.fill(dist, -1);
    Queue<Integer> q = new ArrayDeque<>();

    for (int s : sources) {
        if (dist[s] == -1) {
            dist[s] = 0;
            q.offer(s);
        }
    }

    while (!q.isEmpty()) {
        int u = q.poll();
        for (int v : adj.get(u)) {
            if (dist[v] == -1) {
                dist[v] = dist[u] + 1;
                q.offer(v);
            }
        }
    }
    return dist;
}
```

Interview notes: The returned distance is to the nearest source, not to one particular source.

Part 2 — Cycles, bipartite graphs and ordering

6. Cycle detection in an undirected graph (DFS)

Definition / Intro: A cycle is a closed path that starts and ends at the same vertex without repeating intermediate vertices.

When to use: Check whether an undirected network contains a cycle.

Core idea: During DFS, an already-visited neighbour indicates a cycle only if that neighbour is not the current node's parent.

Complexity: O(V + E) time and O(V) space.

```
static boolean hasUndirectedCycle(int u, int parent,
        List<List<Integer>> adj, boolean[] seen) {
    seen[u] = true;

    for (int v : adj.get(u)) {
        if (!seen[v]) {
            if (hasUndirectedCycle(v, u, adj, seen)) return true;
        } else if (v != parent) {
            return true;
        }
    }
    return false;
}

static boolean containsUndirectedCycle(List<List<Integer>> adj) {
    boolean[] seen = new boolean[adj.size()];
    for (int i = 0; i < adj.size(); i++) {
        if (!seen[i] && hasUndirectedCycle(i, -1, adj, seen)) return true;
    }
    return false;
}
```

Interview notes: The parent-vertex method assumes no parallel edges. If parallel edges are allowed, store edge IDs and skip only the exact edge used to enter the node.

7. Cycle detection in an undirected graph (BFS)

Definition / Intro: BFS can detect an undirected cycle by tracking each node's parent.

When to use: Alternative to recursive DFS, especially when avoiding recursion.

Core idea: If a visited neighbour is not the current node's parent, a cycle exists.

Complexity: O(V + E) time and O(V) space.

```
static boolean containsCycleBfs(List<List<Integer>> adj) {
    int n = adj.size();
    boolean[] seen = new boolean[n];
    int[] parent = new int[n];
    Arrays.fill(parent, -1);

    for (int s = 0; s < n; s++) {
        if (seen[s]) continue;
        Queue<Integer> q = new ArrayDeque<>();
        q.offer(s);
        seen[s] = true;

        while (!q.isEmpty()) {
            int u = q.poll();
            for (int v : adj.get(u)) {
                if (!seen[v]) {
                    seen[v] = true;
                    parent[v] = u;
                    q.offer(v);
                } else if (parent[u] != v) {
                    return true;
                }
            }
        }
    }
    return false;
}
```

Interview notes: Like the parent-vertex DFS, this simple version needs edge IDs to handle parallel edges correctly.

8. Cycle detection in a directed graph (DFS colours)

Definition / Intro: A directed cycle follows edge directions and eventually returns to a vertex already on the current DFS path.

When to use: Dependency-cycle detection, course prerequisites, directed graph validation.

Core idea: Use three states: 0 = unvisited, 1 = currently in recursion path, 2 = fully processed. An edge to state 1 is a back edge and proves a cycle.

Complexity: O(V + E) time and O(V) space.

```
static boolean directedCycleDfs(int u, List<List<Integer>> adj, int[] state) {
    state[u] = 1;

    for (int v : adj.get(u)) {
        if (state[v] == 1) return true;
        if (state[v] == 0 && directedCycleDfs(v, adj, state)) return true;
    }

    state[u] = 2;
    return false;
}

static boolean hasDirectedCycle(List<List<Integer>> adj) {
    int[] state = new int[adj.size()];
    for (int i = 0; i < adj.size(); i++) {
        if (state[i] == 0 && directedCycleDfs(i, adj, state)) return true;
    }
    return false;
}
```

Interview notes: A globally visited array alone is insufficient: an edge to a fully processed vertex does not necessarily form a cycle.

9. Bipartite Graph Check (BFS colouring)

Definition / Intro: A graph is bipartite if its vertices can be divided into two sets so every edge joins different sets.

When to use: Two-team assignment, conflict-free grouping, checking whether an undirected graph has an odd cycle.

Core idea: Colour each component with 0/1. Every edge must connect opposite colours.

Complexity: O(V + E) time and O(V) space.

```
static boolean isBipartite(List<List<Integer>> adj) {
    int n = adj.size();
    int[] color = new int[n];
    Arrays.fill(color, -1);

    for (int s = 0; s < n; s++) {
        if (color[s] != -1) continue;
        Queue<Integer> q = new ArrayDeque<>();
        q.offer(s);
        color[s] = 0;

        while (!q.isEmpty()) {
            int u = q.poll();
            for (int v : adj.get(u)) {
                if (color[v] == -1) {
                    color[v] = 1 - color[u];
                    q.offer(v);
                } else if (color[v] == color[u]) {
                    return false;
                }
            }
        }
    }
    return true;
}
```

Interview notes: An undirected graph is bipartite exactly when it has no odd-length cycle. A self-loop immediately makes it non-bipartite.

10. Topological Sort — Kahn's Algorithm

Definition / Intro: A topological ordering is a linear order of a directed acyclic graph (DAG) in which u appears before v for every edge u→v.

When to use: Course Schedule, task scheduling, build systems, dependency resolution.

Core idea: Compute indegrees. Put all zero-indegree vertices into a queue. Remove one, append it, and reduce the indegree of its outgoing neighbours.

Complexity: O(V + E) time and O(V) space.

```
static List<Integer> topologicalKahn(List<List<Integer>> adj) {
    int n = adj.size();
    int[] indegree = new int[n];

    for (int u = 0; u < n; u++)
        for (int v : adj.get(u)) indegree[v]++;

    Queue<Integer> q = new ArrayDeque<>();
    for (int i = 0; i < n; i++)
        if (indegree[i] == 0) q.offer(i);

    List<Integer> order = new ArrayList<>();
    while (!q.isEmpty()) {
        int u = q.poll();
        order.add(u);
        for (int v : adj.get(u)) {
            if (--indegree[v] == 0) q.offer(v);
        }
    }

    return order.size() == n ? order : new ArrayList<>();
}
```

Interview notes: An empty returned list here means a cycle was detected (assuming n > 0). Multiple valid topological orders may exist.

11. Topological Sort — DFS method

Definition / Intro: DFS topological sorting orders vertices by reverse finishing time.

When to use: Another way to order a DAG; useful when already using DFS-based graph processing.

Core idea: After exploring all outgoing neighbours of u, push u onto a stack. Pop the stack to obtain the order.

Complexity: O(V + E) time and O(V) space.

```
static void topoDfs(int u, List<List<Integer>> adj,
                    boolean[] seen, Deque<Integer> st) {
    seen[u] = true;
    for (int v : adj.get(u))
        if (!seen[v]) topoDfs(v, adj, seen, st);
    st.push(u);
}

static List<Integer> topologicalDfs(List<List<Integer>> adj) {
    int n = adj.size();
    boolean[] seen = new boolean[n];
    Deque<Integer> st = new ArrayDeque<>();
    for (int i = 0; i < n; i++)
        if (!seen[i]) topoDfs(i, adj, seen, st);

    List<Integer> ans = new ArrayList<>();
    while (!st.isEmpty()) ans.add(st.pop());
    return ans;
}
```

Interview notes: This method assumes the graph is a DAG. Add a recursion-stack/colour check if you also need cycle detection.

Part 3 — Shortest paths

12. Unweighted shortest path (BFS + parent reconstruction)

Definition / Intro: In an unweighted graph, BFS finds a path with the minimum number of edges.

When to use: Minimum moves, minimum transformations, shortest route when every edge costs one.

Core idea: Store dist[v] = dist[u] + 1 and parent[v] = u when v is first discovered.

Complexity: O(V + E) time and O(V) space.

```
static List<Integer> shortestUnweightedPath(
        List<List<Integer>> adj, int src, int dest) {
    int n = adj.size();
    int[] dist = new int[n], parent = new int[n];
```

```
    Arrays.fill(dist, -1);
    Arrays.fill(parent, -1);

    Queue<Integer> q = new ArrayDeque<>();
    q.offer(src);
    dist[src] = 0;

    while (!q.isEmpty()) {
        int u = q.poll();
        if (u == dest) break;
        for (int v : adj.get(u)) {
            if (dist[v] == -1) {
                dist[v] = dist[u] + 1;
                parent[v] = u;
                q.offer(v);
            }
        }
    }

    if (dist[dest] == -1) return Collections.emptyList();
    List<Integer> path = new ArrayList<>();
    for (int x = dest; x != -1; x = parent[x]) path.add(x);
    Collections.reverse(path);
    return path;
}
```

Interview notes: The path includes both source and destination. An empty list means unreachable.

13. Dijkstra's Algorithm

Definition / Intro: Dijkstra computes shortest distances from one source when all edge weights are non-negative.

When to use: Weighted shortest paths with no negative edge weights.

Core idea: Use a min-priority queue. Relax an edge if dist[u] + weight improves dist[v]. Ignore stale queue entries.

Complexity: With adjacency list + binary heap: O((V + E) log V) time; O(V + E) storage.

```
static class Edge {
    int to;
    long weight;
    Edge(int to, long weight) { this.to = to; this.weight = weight; }
}
static class State implements Comparable<State> {
    int node; long dist;
    State(int node, long dist) { this.node = node; this.dist = dist; }
    public int compareTo(State other) {
        return Long.compare(this.dist, other.dist);
    }
}

static long[] dijkstra(List<List<Edge>> adj, int src) {
    int n = adj.size();
    long INF = Long.MAX_VALUE / 4;
    long[] dist = new long[n];
    Arrays.fill(dist, INF);
    PriorityQueue<State> pq = new PriorityQueue<>();

    dist[src] = 0;
    pq.offer(new State(src, 0));

    while (!pq.isEmpty()) {
        State cur = pq.poll();
        int u = cur.node;
        if (cur.dist != dist[u]) continue; // stale entry

        for (Edge e : adj.get(u)) {
            if (e.weight < 0)
                throw new IllegalArgumentException("Dijkstra needs non-negative weights");
            long nd = dist[u] + e.weight;
            if (nd < dist[e.to]) {
                dist[e.to] = nd;
                pq.offer(new State(e.to, nd));
            }
        }
    }
    return dist;
}
```

Interview notes: Never use Dijkstra when negative edges are present. Use long for distances. Unreachable vertices retain INF.

14. Bellman–Ford Algorithm

Definition / Intro: Bellman–Ford finds single-source shortest paths even when negative edge weights exist, and can detect a source-reachable negative cycle.

When to use: Negative edges, currency/arbitrage-style modelling, or when Dijkstra is invalid.

Core idea: Relax every edge V−1 times. A further successful relaxation means a reachable negative cycle.

Complexity: O(VE) time and O(V) distance storage.

```
static class WEdge {
    int from, to; long weight;
    WEdge(int f, int t, long w) { from=f; to=t; weight=w; }
}

static long[] bellmanFord(int n, List<WEdge> edges, int src) {
    long INF = Long.MAX_VALUE / 4;
    long[] dist = new long[n];
    Arrays.fill(dist, INF);
    dist[src] = 0;

    for (int i = 1; i <= n - 1; i++) {
        boolean changed = false;
        for (WEdge e : edges) {
            if (dist[e.from] != INF &&
                dist[e.from] + e.weight < dist[e.to]) {
                dist[e.to] = dist[e.from] + e.weight;
                changed = true;
            }
        }
        if (!changed) break;
    }

    for (WEdge e : edges) {
        if (dist[e.from] != INF &&
            dist[e.from] + e.weight < dist[e.to]) {
            throw new IllegalStateException("Reachable negative cycle");
        }
    }
    return dist;
}
```

Interview notes: The check above detects only a negative cycle reachable from src. Use a super-source or initialise all distances to 0 to detect a negative cycle anywhere.

15. Floyd–Warshall Algorithm

Definition / Intro: Floyd–Warshall computes shortest distances between every pair of vertices.

When to use: All-pairs shortest paths on small/dense graphs; negative edges are allowed, but negative cycles make shortest paths undefined.

Core idea: Try every vertex k as an intermediate point: dist[i][j] = min(dist[i][j], dist[i][k] + dist[k][j]).

Complexity: O(V^3) time and O(V^2) space.

```
static void floydWarshall(long[][] d, long INF) {
    int n = d.length;
    for (int k = 0; k < n; k++) {
        for (int i = 0; i < n; i++) {
            if (d[i][k] == INF) continue;
            for (int j = 0; j < n; j++) {
                if (d[k][j] == INF) continue;
                d[i][j] = Math.min(d[i][j], d[i][k] + d[k][j]);
            }
        }
    }
    // A negative d[i][i] after the algorithm indicates a negative cycle.
}
```

Interview notes: Initialise d[i][i] = 0; put the minimum direct edge weight in d[u][v]; use INF for no edge. Check for negative diagonal entries afterward.

16. 0–1 BFS

Definition / Intro: 0–1 BFS is a shortest-path algorithm for graphs whose edge weights are only 0 or 1.

When to use: Minimum cost when transitions are free or cost one; problems such as minimum edge reversals.

Core idea: Use a deque. For weight 0, push the neighbour to the front; for weight 1, push it to the back.

Complexity: O(V + E) time for 0/1 edge weights.

```
static long[] zeroOneBfs(List<List<Edge>> adj, int src) {
    int n = adj.size();
    long INF = Long.MAX_VALUE / 4;
    long[] dist = new long[n];
    Arrays.fill(dist, INF);
    Deque<Integer> dq = new ArrayDeque<>();

    dist[src] = 0;
    dq.addFirst(src);

    while (!dq.isEmpty()) {
        int u = dq.removeFirst();
        for (Edge e : adj.get(u)) {
            if (e.weight != 0 && e.weight != 1)
                throw new IllegalArgumentException("Weights must be 0 or 1");
            long nd = dist[u] + e.weight;
            if (nd < dist[e.to]) {
                dist[e.to] = nd;
                if (e.weight == 0) dq.addFirst(e.to);
                else dq.addLast(e.to);
            }
        }
    }
    return dist;
}
```

Interview notes: Do not replace the deque with a normal FIFO queue. The front/back placement is what preserves the 0–1 ordering.

17. DAG shortest path

Definition / Intro: A DAG shortest-path algorithm relaxes edges in topological order; edge weights may be negative because a DAG has no cycles.

When to use: Weighted dependency graphs, shortest paths in a DAG, scheduling with costs.

Core idea: Topologically sort vertices, then relax every outgoing edge from each vertex in that order.

Complexity: O(V + E) time and O(V) distance storage.

```
static long[] dagShortestPath(List<List<Edge>> adj, int src) {
    int n = adj.size();
    List<Integer> order = topologicalKahnWeighted(adj); // helper below
    long INF = Long.MAX_VALUE / 4;
    long[] dist = new long[n];
    Arrays.fill(dist, INF);
    dist[src] = 0;

    for (int u : order) {
        if (dist[u] == INF) continue;
        for (Edge e : adj.get(u))
            dist[e.to] = Math.min(dist[e.to], dist[u] + e.weight);
    }
    return dist;
}

static List<Integer> topologicalKahnWeighted(List<List<Edge>> adj) {
    int n = adj.size();
    int[] indeg = new int[n];
    for (int u = 0; u < n; u++)
        for (Edge e : adj.get(u)) indeg[e.to]++;

    Queue<Integer> q = new ArrayDeque<>();
```

```
    for (int i = 0; i < n; i++) if (indeg[i] == 0) q.offer(i);
    List<Integer> order = new ArrayList<>();

    while (!q.isEmpty()) {
        int u = q.poll(); order.add(u);
        for (Edge e : adj.get(u))
            if (--indeg[e.to] == 0) q.offer(e.to);
    }
    if (order.size() != n) throw new IllegalArgumentException("Not a DAG");
    return order;
}
```

Interview notes: Unlike Dijkstra, this supports negative weights. For longest path in a DAG, replace Math.min with Math.max and initialise unreachable values appropriately.

18. Johnson's Algorithm (advanced all-pairs shortest paths)

Definition / Intro: Johnson's algorithm computes all-pairs shortest paths efficiently on sparse graphs that may contain negative edges but no negative cycles.

When to use: All-pairs shortest paths on sparse weighted graphs where Floyd–Warshall's O(V^3) is too expensive.

Core idea: Add a super-source, run Bellman–Ford to obtain vertex potentials, reweight edges to non-negative weights, then run Dijkstra from each vertex and convert distances back.

Complexity: Typically O(VE + V(E log V)) with a binary heap; exact bounds depend on graph representation and heap.

```
// Interview implementation outline:
// 1) Add super-source S with 0-weight edges to every vertex.
// 2) Run Bellman-Ford from S to obtain h[v].
// 3) If Bellman-Ford detects a negative cycle, stop.
// 4) Reweight each edge u->v:
//       w'(u,v) = w(u,v) + h[u] - h[v]   (now non-negative)
// 5) Run Dijkstra from each original vertex using w'.
// 6) Convert back:
//       originalDist(s,t) = reweightedDist(s,t) - h[s] + h[t].
```

Interview notes: Know the reweighting formula and why it works. In interviews, explain the pipeline clearly; implement only if constraints justify it.

Part 4 — Minimum spanning trees and disjoint sets

19. Disjoint Set Union (DSU / Union-Find)

Definition / Intro: DSU maintains a partition of elements into disjoint sets and supports merging sets and checking whether two elements belong to the same set.

When to use: Kruskal MST, dynamic connectivity under edge additions, cycle detection in undirected edge lists.

Core idea: Use path compression in find and union by size/rank in union. These make operations almost constant amortised time.

Complexity: Amortised O(α(V)) per operation, where α is the inverse Ackermann function.

```
static class DSU {
    int[] parent, size;
    DSU(int n) {
        parent = new int[n];
        size = new int[n];
        for (int i = 0; i < n; i++) {
            parent[i] = i; size[i] = 1;
        }
    }
    int find(int x) {
        if (parent[x] != x) parent[x] = find(parent[x]);
        return parent[x];
    }
```

```
    boolean union(int a, int b) {
        a = find(a); b = find(b);
        if (a == b) return false;
        if (size[a] < size[b]) { int t=a; a=b; b=t; }
        parent[b] = a;
        size[a] += size[b];
        return true;
    }
}
```

Interview notes: union returns false when both vertices were already in the same component. This is useful for detecting whether an edge closes a cycle.

20. Kruskal's Minimum Spanning Tree

Definition / Intro: A minimum spanning tree (MST) is a minimum-total-weight set of edges connecting all vertices of a connected, undirected, weighted graph without cycles.

When to use: MST when edges can be sorted; sparse graphs; network design.

Core idea: Sort edges by weight. Add an edge only when its endpoints are in different DSU components.

Complexity: O(E log E) time for sorting; DSU work is near-linear.

```
static class MEdge implements Comparable<MEdge> {
    int u, v; long w;
    MEdge(int u, int v, long w) { this.u=u; this.v=v; this.w=w; }
    public int compareTo(MEdge other) { return Long.compare(w, other.w); }
}

static long kruskal(int n, List<MEdge> edges) {
    edges.sort(null);
    DSU dsu = new DSU(n);
    long total = 0;
    int used = 0;

    for (MEdge e : edges) {
        if (dsu.union(e.u, e.v)) {
            total += e.w;
            used++;
            if (used == n - 1) break;
        }
    }
    if (used != n - 1) throw new IllegalArgumentException("Graph is disconnected");
    return total;
}
```

Interview notes: For a disconnected graph, the same process returns a minimum spanning forest rather than one MST. Negative edge weights are allowed.

21. Prim's Minimum Spanning Tree

Definition / Intro: Prim grows an MST from a starting vertex by repeatedly choosing the minimum-weight edge that connects the current tree to a new vertex.

When to use: MST on adjacency-list graphs; often convenient when the graph is already stored as adjacency lists.

Core idea: Use a min-priority queue of candidate edges. Skip vertices already included in the tree.

Complexity: O((V + E) log V) with adjacency list + binary heap.

```
static long prim(List<List<Edge>> adj) {
    int n = adj.size();
    boolean[] used = new boolean[n];
    PriorityQueue<State> pq = new PriorityQueue<>();
    pq.offer(new State(0, 0)); // node 0, edge cost 0
    long total = 0;
    int count = 0;

    while (!pq.isEmpty()) {
        State cur = pq.poll();
        int u = cur.node;
        if (used[u]) continue;
        used[u] = true;
        total += cur.dist;
        count++;
```

```
        for (Edge e : adj.get(u))
            if (!used[e.to]) pq.offer(new State(e.to, e.weight));
    }

    if (count != n) throw new IllegalArgumentException("Graph is disconnected");
    return total;
}
```

Interview notes: This version assumes a connected graph and uses the State class from Dijkstra. For a minimum spanning forest, start Prim from every unvisited vertex and track each component separately.

Part 5 — Strong connectivity and graph structure

22. Strongly Connected Components (SCC) — Kosaraju

Definition / Intro: An SCC is a maximal set of vertices in a directed graph where every vertex can reach every other vertex.

When to use: Condensation graphs, dependency analysis, mutual reachability, 2-SAT foundations.

Core idea: First DFS records finishing order. Reverse all edges. Process vertices in reverse finishing order on the reversed graph; each DFS tree is one SCC.

Complexity: O(V + E) time and O(V + E) space.

```
static void finishDfs(int u, List<List<Integer>> g,
                      boolean[] seen, List<Integer> order) {
    seen[u] = true;
    for (int v : g.get(u)) if (!seen[v]) finishDfs(v, g, seen, order);
    order.add(u);
}

static void collectDfs(int u, List<List<Integer>> rg,
                       boolean[] seen, List<Integer> component) {
    seen[u] = true;
    component.add(u);
    for (int v : rg.get(u)) if (!seen[v]) collectDfs(v, rg, seen, component);
}

static List<List<Integer>> kosaraju(List<List<Integer>> g) {
    int n = g.size();
    List<List<Integer>> rg = new ArrayList<>();
    for (int i=0; i<n; i++) rg.add(new ArrayList<>());
    for (int u=0; u<n; u++)
        for (int v : g.get(u)) rg.get(v).add(u);

    boolean[] seen = new boolean[n];
    List<Integer> order = new ArrayList<>();
    for (int i=0; i<n; i++) if (!seen[i]) finishDfs(i,g,seen,order);

    Arrays.fill(seen, false);
    Collections.reverse(order);
    List<List<Integer>> result = new ArrayList<>();
    for (int u : order) {
        if (!seen[u]) {
            List<Integer> comp = new ArrayList<>();
            collectDfs(u,rg,seen,comp);
            result.add(comp);
        }
    }
    return result;
}
```

Interview notes: Kosaraju needs the reversed graph. For very deep graphs, convert recursive DFS to iterative form.

23. Strongly Connected Components — Tarjan

Definition / Intro: Tarjan's SCC algorithm finds all strongly connected components in one DFS traversal using discovery times and low-link values.

When to use: SCC when building the reversed graph is undesirable.

Core idea: Maintain a DFS stack and onStack[]. low[u] tracks the earliest discovery time reachable from u through DFS-tree edges and back edges. When low[u] == disc[u], u roots an SCC.

Complexity: O(V + E) time and O(V) auxiliary space.

```
static class TarjanSCC {
    List<List<Integer>> g, comps = new ArrayList<>();
    int[] disc, low;
    boolean[] onStack;
    Deque<Integer> st = new ArrayDeque<>();
    int time = 0;

    TarjanSCC(List<List<Integer>> graph) {
        g = graph;
        int n = g.size();
        disc = new int[n]; low = new int[n]; onStack = new boolean[n];
        Arrays.fill(disc, -1);
        for (int i=0; i<n; i++) if (disc[i] == -1) dfs(i);
    }

    void dfs(int u) {
        disc[u] = low[u] = time++;
        st.push(u); onStack[u] = true;

        for (int v : g.get(u)) {
            if (disc[v] == -1) {
                dfs(v);
                low[u] = Math.min(low[u], low[v]);
            } else if (onStack[v]) {
                low[u] = Math.min(low[u], disc[v]);
            }
        }

        if (low[u] == disc[u]) {
            List<Integer> comp = new ArrayList<>();
            int x;
            do {
                x = st.pop();
                onStack[x] = false;
                comp.add(x);
            } while (x != u);
            comps.add(comp);
        }
    }
}
```

Interview notes: Do not confuse Tarjan SCC low-link updates with bridge-finding low-link updates; the visited-neighbour rule differs.

24. Bridges (critical connections)

Definition / Intro: A bridge is an undirected edge whose removal increases the number of connected components.

When to use: Critical Connections in a Network; reliability analysis; identifying single points of failure.

Core idea: DFS assigns discovery time disc[u] and low[u]. Tree edge u-v is a bridge if low[v] > disc[u].

Complexity: O(V + E) time and O(V) auxiliary space.

```
static class BridgeFinder {
    List<List<int[]>> g; // each item: {neighbor, edgeId}
    int timer = 0;
    int[] tin, low;
    List<int[]> bridges = new ArrayList<>();

    BridgeFinder(int n, List<int[]> edges) {
        g = new ArrayList<>();
        for (int i=0; i<n; i++) g.add(new ArrayList<>());
        for (int id=0; id<edges.size(); id++) {
            int u=edges.get(id)[0], v=edges.get(id)[1];
            g.get(u).add(new int[]{v,id});
            g.get(v).add(new int[]{u,id});
        }
        tin = new int[n]; low = new int[n];
        Arrays.fill(tin, -1);
        for (int i=0; i<n; i++) if (tin[i] == -1) dfs(i,-1);
    }
```

```
    void dfs(int u, int parentEdge) {
        tin[u] = low[u] = timer++;
        for (int[] item : g.get(u)) {
            int v=item[0], id=item[1];
            if (id == parentEdge) continue;
            if (tin[v] != -1) {
                low[u] = Math.min(low[u], tin[v]);
            } else {
                dfs(v,id);
                low[u] = Math.min(low[u], low[v]);
                if (low[v] > tin[u]) bridges.add(new int[]{u,v});
            }
        }
    }
}
```

Interview notes: Edge IDs correctly handle parallel edges. Run DFS from every vertex to handle disconnected graphs.

25. Articulation Points

Definition / Intro: An articulation point (cut vertex) is a vertex whose removal increases the number of connected components.

When to use: Find critical routers, servers, or vertices in an undirected network.

Core idea: For a DFS tree root, it is an articulation point if it has more than one DFS child. A non-root u is one if some child v has low[v] >= disc[u].

Complexity: O(V + E) time and O(V) space.

```
static class CutVertices {
    List<List<Integer>> g;
    int[] tin, low;
    boolean[] cut;
    int timer = 0;

    CutVertices(List<List<Integer>> graph) {
        g = graph;
        int n = g.size();
        tin = new int[n]; low = new int[n]; cut = new boolean[n];
        Arrays.fill(tin,-1);
        for (int i=0; i<n; i++) if (tin[i] == -1) dfs(i,-1);
    }

    void dfs(int u, int parent) {
        tin[u] = low[u] = timer++;
        int children = 0;

        for (int v : g.get(u)) {
            if (v == parent) continue;
            if (tin[v] != -1) {
                low[u] = Math.min(low[u], tin[v]);
            } else {
                dfs(v,u);
                low[u] = Math.min(low[u], low[v]);
                children++;
                if (parent != -1 && low[v] >= tin[u]) cut[u] = true;
            }
        }
        if (parent == -1 && children > 1) cut[u] = true;
    }
}
```

Interview notes: For multigraph-safe production code, store edge IDs and skip only the exact parent edge, as in the bridge implementation.

Part 6 — Advanced graph patterns

26. Clone Graph

Definition / Intro: Graph cloning creates a deep copy: every original node gets a new node, and copied edges connect copied nodes.

When to use: Copy an arbitrary connected graph with cycles and possible self-loops.

Core idea: Use a map from original node to clone. BFS/DFS ensures each original node is cloned once.

Complexity: O(V + E) time and O(V) space.

```
class Node {
    int val;
    List<Node> neighbors;
    Node(int val) { this.val = val; neighbors = new ArrayList<>(); }
}

static Node cloneGraph(Node start) {
    if (start == null) return null;
    Map<Node,Node> map = new HashMap<>();
    Queue<Node> q = new ArrayDeque<>();

    map.put(start, new Node(start.val));
    q.offer(start);

    while (!q.isEmpty()) {
        Node cur = q.poll();
        for (Node nei : cur.neighbors) {
            if (!map.containsKey(nei)) {
                map.put(nei, new Node(nei.val));
                q.offer(nei);
            }
            map.get(cur).neighbors.add(map.get(nei));
        }
    }
    return map.get(start);
}
```

Interview notes: The map is essential because graphs may contain cycles. Do not use node values as unique keys unless the problem guarantees uniqueness.

27. Bipartite Matching — Kuhn's Algorithm

Definition / Intro: A matching is a set of edges with no shared endpoints. In bipartite matching, edges go only between a left set and a right set.

When to use: Assign workers to jobs, students to projects, or applicants to positions when each can be matched at most once.

Core idea: Try to assign each left vertex a right partner. If occupied, recursively try to move its current partner elsewhere (augmenting path).

Complexity: O(VE) for the standard Kuhn implementation.

```
static boolean tryMatch(int u, List<List<Integer>> g,
                        boolean[] seenRight, int[] matchRight) {
    for (int v : g.get(u)) {
        if (seenRight[v]) continue;
        seenRight[v] = true;

        if (matchRight[v] == -1 ||
            tryMatch(matchRight[v], g, seenRight, matchRight)) {
            matchRight[v] = u;
            return true;
        }
    }
    return false;
}

static int maxBipartiteMatching(List<List<Integer>> leftToRight, int rightN) {
    int[] matchRight = new int[rightN];
    Arrays.fill(matchRight, -1);
    int answer = 0;

    for (int u=0; u<leftToRight.size(); u++) {
        boolean[] seenRight = new boolean[rightN];
        if (tryMatch(u,leftToRight,seenRight,matchRight)) answer++;
    }
    return answer;
}
```

Interview notes: For larger bipartite graphs, Hopcroft–Karp improves the time to O(E√V).

28. Hopcroft–Karp Matching

Definition / Intro: Hopcroft–Karp finds a maximum-cardinality matching in a bipartite graph by finding many shortest augmenting paths per phase.

When to use: Large bipartite matching instances where Kuhn's O(VE) may be too slow.

Core idea: BFS builds layers of shortest augmenting paths; DFS finds a maximal set of vertex-disjoint shortest augmenting paths in that phase.

Complexity: O(E√V) time.

```
// Interview-level algorithm outline:
// 1) Keep pairU[u] and pairV[v], initially unmatched.
// 2) BFS from every unmatched left vertex to build distance layers.
// 3) DFS only follows edges that advance one layer.
// 4) Augment along found paths; repeat until no augmenting path exists.
// This is a larger implementation; be able to explain the BFS/DFS phases
// and implement Kuhn first unless constraints demand Hopcroft-Karp.
```

Interview notes: Know why it is faster than Kuhn: it augments along multiple shortest paths in one phase.

29. Maximum Flow — Edmonds–Karp

Definition / Intro: A flow network is a directed graph with capacities. Maximum flow is the greatest amount that can be sent from source s to sink t without exceeding capacities or violating flow conservation.

When to use: Network bandwidth, assignment/transport models, edge-disjoint paths, min-cut problems.

Core idea: Repeatedly find an augmenting path in the residual graph using BFS. Push the bottleneck capacity along that path.

Complexity: O(VE^2) time.

```
static long edmondsKarp(long[][] cap, int s, int t) {
    int n = cap.length;
    long flow = 0;

    while (true) {
        int[] parent = new int[n];
        Arrays.fill(parent,-1);
        parent[s] = s;
        Queue<Integer> q = new ArrayDeque<>();
        q.offer(s);

        while (!q.isEmpty() && parent[t] == -1) {
            int u = q.poll();
            for (int v=0; v<n; v++) {
                if (parent[v] == -1 && cap[u][v] > 0) {
                    parent[v] = u;
                    q.offer(v);
                }
            }
        }
        if (parent[t] == -1) break;

        long add = Long.MAX_VALUE;
        for (int v=t; v!=s; v=parent[v])
            add = Math.min(add,cap[parent[v]][v]);

        for (int v=t; v!=s; v=parent[v]) {
            int u=parent[v];
            cap[u][v] -= add;
            cap[v][u] += add;
        }
        flow += add;
    }
    return flow;
}
```

Interview notes: This matrix implementation is suitable for small/dense graphs and aggregates capacities by ordered pair. For sparse graphs, use adjacency lists with explicit reverse-edge indices.

30. Dinic's Maximum Flow

Definition / Intro: Dinic's algorithm computes maximum flow using a level graph and blocking flows.

When to use: Larger maximum-flow problems where Edmonds–Karp is too slow.

Core idea: BFS builds levels in the residual graph. DFS sends flow only along level-increasing edges until no more blocking flow can be sent; repeat.

Complexity: General bound O(V^2E); often faster in practice and has stronger bounds on special graph classes.

```
// Core implementation structure:
// buildLevelGraph(): BFS over residual edges; level[v] = level[u] + 1.
// sendFlow(u, pushed): DFS only when level[to] == level[u] + 1.
// Use current-edge pointer ptr[u] so failed edges are not rescanned.
// Repeat sendFlow(source, INF) until it returns 0; rebuild levels.
```

Interview notes: For interviews, explain residual edges, reverse capacities, level graph, blocking flow, and current-edge optimisation.

31. Eulerian Path and Eulerian Circuit

Definition / Intro: An Eulerian path uses every edge exactly once. An Eulerian circuit uses every edge exactly once and returns to its start.

When to use: Route inspection, drawing a graph without lifting a pen, traversing every road/edge once.

Core idea: For undirected connected non-isolated vertices: circuit requires all degrees even; path requires exactly 0 or 2 odd-degree vertices. Hierholzer constructs the route.

Complexity: O(V + E) time with adjacency lists and edge-visited tracking.

```
// Hierholzer outline (undirected multigraph-safe):
// Store each adjacency entry as {neighbor, edgeId}.
// Maintain used[edgeId] so each undirected edge is consumed once.
// Push start on a stack.
// While stack not empty:
//   if top has an unused incident edge, mark its edgeId used and push neighbor.
//   otherwise pop top and append it to answer.
// Reverse answer at the end.
```

Interview notes: For directed graphs, all non-isolated vertices must belong to one weakly connected component; circuit additionally requires inDegree==outDegree for every vertex. A directed trail has one vertex with out=in+1, one with in=out+1, and all others balanced.

32. Hamiltonian Path and Hamiltonian Cycle

Definition / Intro: A Hamiltonian path visits every vertex exactly once. A Hamiltonian cycle does so and returns to the starting vertex.

When to use: Small-constraint route/tour problems where every vertex must be visited exactly once.

Core idea: Backtracking tries an unused adjacent vertex; undo the choice if it does not lead to a complete path.

Complexity: Worst-case O(V!) time for basic backtracking; exponential.

```
static boolean hamDfs(int u, List<List<Integer>> g,
                      boolean[] used, int count) {
    if (count == g.size()) return true;

    for (int v : g.get(u)) {
        if (!used[v]) {
            used[v] = true;
            if (hamDfs(v,g,used,count+1)) return true;
            used[v] = false;
        }
    }
    return false;
}

static boolean hasHamiltonianPath(List<List<Integer>> g) {
    int n = g.size();
    for (int start=0; start<n; start++) {
        boolean[] used = new boolean[n];
        used[start] = true;
        if (hamDfs(start,g,used,1)) return true;
```

```
    }
    return n == 0;
}
```

Interview notes: This checks a Hamiltonian path, not a cycle. Hamiltonian cycle additionally checks whether the final vertex has an edge back to the chosen start.

33. A* Search

Definition / Intro: A* is a best-first pathfinding algorithm that prioritises nodes using f(n) = g(n) + h(n), where g is known cost so far and h estimates remaining cost.

When to use: Pathfinding on maps/grids, games, robotics, when a useful admissible heuristic is available.

Core idea: Expand the state with the smallest estimated total cost. An admissible heuristic never overestimates the true remaining cost; consistency is an even stronger useful property.

Complexity: Depends on heuristic and graph; worst-case exponential in search depth, with priority-queue overhead.

```
// Core priority:
// f = g + h
// PQ ordered by f; keep bestG[state].
// When a better g is found, update bestG and push a new entry.
// Ignore stale entries when popped.
// With h(n)=0, A* reduces to Dijkstra.
```

Interview notes: Do not claim A* is always faster than Dijkstra. Its benefit depends on a good heuristic and correct state modelling.

34. Bidirectional BFS

Definition / Intro: Bidirectional BFS searches forward from the source and backward from the target, stopping when the search frontiers meet.

When to use: Shortest path in a large unweighted graph when both endpoints are known and reverse traversal is possible.

Core idea: Expand the smaller frontier at each step; track distances from both ends and detect an intersection.

Complexity: Often reduces explored states dramatically; worst-case remains O(V + E).

```
// Core pattern:
// distS[src]=0; distT[target]=0
// frontierS={src}; frontierT={target}
// Expand one full BFS layer from the smaller frontier.
// If a neighbour has a distance from the opposite side, combine distances.
// For directed graphs, reverse edges are needed for the backward search.
```

Interview notes: It is not appropriate when the destination is unknown or when reverse transitions cannot be generated.

35. 2-SAT (Implication Graph)

Definition / Intro: 2-SAT decides whether a Boolean formula made of clauses (a OR b) is satisfiable, where each clause has two literals.

When to use: Boolean constraint problems, scheduling with binary choices, logical implication systems.

Core idea: For each clause (a OR b), add implications (NOT a → b) and (NOT b → a). The formula is satisfiable iff no variable and its negation lie in the same SCC.

Complexity: O(V + E) time using Kosaraju or Tarjan; graph has 2N literal vertices.

```
// Literal indexing idea:
// variable x: node 2*x
// negation !x: node 2*x + 1
// neg(node) = node ^ 1
// Clause (a OR b):
// addEdge(a ^ 1, b)
// addEdge(b ^ 1, a)
```

```
// After SCC:
// if comp[2*x] == comp[2*x + 1], UNSAT.
// Otherwise assign values using SCC topological order.
```

Interview notes: The most error-prone part is consistent literal encoding and assignment order. Practise satisfiability first, then assignment reconstruction.

36. Tree diameter (two BFS/DFS sweeps)

Definition / Intro: The diameter of a tree is the maximum distance between any pair of vertices.

When to use: Longest path in a tree, tree-centre problems, network span.

Core idea: Run BFS from any vertex to find a farthest vertex A. Run BFS from A; the maximum distance found is the diameter.

Complexity: O(V + E), which is O(V) for a tree.

```
static int[] bfsDistances(List<List<Integer>> tree, int src) {
    int[] d = new int[tree.size()];
    Arrays.fill(d,-1);
    Queue<Integer> q = new ArrayDeque<>();
    q.offer(src); d[src]=0;
    while (!q.isEmpty()) {
        int u=q.poll();
        for (int v : tree.get(u)) if (d[v] == -1) {
            d[v]=d[u]+1; q.offer(v);
        }
    }
    return d;
}

static int treeDiameter(List<List<Integer>> tree) {
    int[] d0 = bfsDistances(tree,0);
    int a=0;
    for (int i=0;i<d0.length;i++) if (d0[i]>d0[a]) a=i;
    int[] da = bfsDistances(tree,a);
    int diameter=0;
    for (int x : da) diameter=Math.max(diameter,x);
    return diameter;
}
```

Interview notes: This simple version assumes a non-empty tree and unweighted edges. For weighted trees, use a traversal accumulating path weights.

Part 7 — Tree-related graph algorithms (important interview extension)

Trees are connected acyclic graphs. Many interview problems call these 'tree problems' rather than graph problems, but they share adjacency-list traversal and are commonly included in graph preparation.

37. Lowest Common Ancestor (LCA) — Binary Lifting

Definition / Intro: The LCA of nodes u and v is their deepest common ancestor in a rooted tree.

When to use: Repeated ancestor queries, distance between tree nodes, path queries.

Core idea: Precompute up[k][v], the 2^k-th ancestor of v. Lift the deeper node, then lift both nodes from largest power downward.

Complexity: Preprocessing O(V log V); each query O(log V); space O(V log V).

```
static class LCA {
    int LOG;
    int[][] up;
    int[] depth;
    List<List<Integer>> g;

    LCA(List<List<Integer>> tree, int root) {
        g=tree;
        int n=g.size();
        LOG=1;
        while ((1<<LOG) <= n) LOG++;
```

```
        up=new int[LOG][n];
        depth=new int[n];
        for (int[] row:up) Arrays.fill(row,-1);
        build(root,-1);
        for (int k=1;k<LOG;k++)
            for (int v=0;v<n;v++)
                if (up[k-1][v] != -1)
                    up[k][v]=up[k-1][up[k-1][v]];
    }

    void build(int u,int p) {
        up[0][u]=p;
        for (int v:g.get(u)) if (v!=p) {
            depth[v]=depth[u]+1;
            build(v,u);
        }
    }

    int query(int a,int b) {
        if (depth[a]<depth[b]) { int t=a; a=b; b=t; }
        int diff=depth[a]-depth[b];
        for (int k=0;k<LOG;k++)
            if (((diff>>k)&1)==1) a=up[k][a];
        if (a==b) return a;
        for (int k=LOG-1;k>=0;k--)
            if (up[k][a]!=up[k][b]) {
                a=up[k][a]; b=up[k][b];
            }
        return up[0][a];
    }
}
```

Interview notes: This code assumes a connected tree and a valid root. For very deep trees, make the preprocessing traversal iterative.

38. Euler Tour of a Tree

Definition / Intro: An Euler tour records entry/exit times during DFS and converts subtree relationships into array intervals.

When to use: Subtree sum/count queries, subtree range updates, ancestor checks.

Core idea: On entering u, assign tin[u]; after all children, assign tout[u]. A vertex v is in u's subtree when tin[u] <= tin[v] < tout[u].

Complexity: O(V) preprocessing; subtree becomes a contiguous interval in Euler order.

```
static int timer = 0;
static void eulerDfs(int u,int p,List<List<Integer>> g,
                     int[] tin,int[] tout,int[] order) {
    tin[u]=timer;
    order[timer++]=u;
    for (int v:g.get(u)) if (v!=p)
        eulerDfs(v,u,g,tin,tout,order);
    tout[u]=timer;
}
```

Interview notes: This is often combined with Fenwick Tree or Segment Tree. Be clear whether your convention uses inclusive or exclusive tout.

39. Heavy-Light Decomposition (HLD)

Definition / Intro: HLD decomposes a tree into heavy paths so a path query can be split into O(log V) contiguous segments.

When to use: Path sum/min/max queries and updates on a tree, usually with a segment tree.

Core idea: Compute subtree sizes and choose the largest child as heavy. Assign each heavy chain consecutive positions in a base array.

Complexity: O(log^2 V) per path query with a segment tree; O(log V) chain jumps.

```
// Implementation blueprint:
// 1) DFS1: parent[], depth[], subtreeSize[], heavyChild[].
// 2) DFS2: head[], position[], baseArray[].
// 3) Build a segment tree over baseArray.
```

```
// 4) While head[u] != head[v], process the deeper chain segment,
//    then move u/v to parent[head[u/head[v]]].
// 5) Process the final segment on the same chain.
```

Interview notes: HLD is an advanced tree technique. Learn subtree sizes and segment trees first; then implement HLD for path-query constraints.

40. Centroid Decomposition

Definition / Intro: Centroid decomposition recursively chooses a centroid whose removal splits a tree into components of size at most half.

When to use: Dynamic distance queries, nearest active node, counting pairs by distance.

Core idea: Find centroid, process information involving it, remove it conceptually, and recurse on each remaining component.

Complexity: Typically O(V log V) decomposition depth; total complexity depends on the query/data structure layered on top.

```
// Core outline:
// 1) Compute component subtree sizes.
// 2) Find a vertex whose every remaining component has size <= N/2.
// 3) Mark it as centroid and process queries/answers through it.
// 4) Recurse independently into each component after removing centroid.
```

Interview notes: This is a specialised advanced topic; interview frequency is lower than BFS/DFS, Dijkstra, MST, SCC, and LCA.

Part 8 — Complexity and algorithm selection

Problem condition	Use this algorithm	Key restriction
Unweighted shortest path	BFS	Equal edge cost
Weights are 0 or 1	0–1 BFS	Only 0/1 weights
Non-negative weights	Dijkstra	No negative edges
Negative edges, one source	Bellman–Ford	Detect reachable negative cycles
All pairs, small/dense graph	Floyd–Warshall	O(V^3)
DAG shortest path	Topological relaxation	Must be acyclic; negative weights okay
Sparse all-pairs, negative edges	Johnson	No negative cycle
Minimum spanning tree	Kruskal / Prim	Undirected weighted graph
Dynamic connectivity under unions	DSU	No arbitrary deletions
Directed mutual reachability	Kosaraju / Tarjan SCC	Directed graph
Critical edge / vertex	Bridges / articulation points	Undirected graph
Bipartite maximum matching	Kuhn / Hopcroft–Karp	Bipartite graph
Maximum flow	Edmonds–Karp / Dinic	Capacitated directed network
Visit every edge once	Hierholzer	Eulerian degree/connectivity conditions
Visit every vertex once	Backtracking / specialised methods	Hamiltonian is hard in general
Tree ancestor queries	LCA binary lifting	Rooted tree

Part 9 — Interview traps and correctness checklist

• Wrong shortest-path algorithm: BFS for unweighted; Dijkstra for non-negative weights; Bellman–Ford for negative edges; DAG relaxation for DAGs.

• Visited timing: in BFS, mark visited when enqueuing.

• Directed vs undirected cycle: parent check is for undirected graphs; recursion-stack/colour state is for directed graphs.

• Disconnected graphs: start traversal from every unvisited vertex when the task asks about the whole graph.

• Parallel edges: use edge IDs for bridge and multigraph-safe cycle logic.

• Overflow: use long for weighted path sums and flow totals where needed.

• Topological sort: exists only for DAGs; if Kahn processes fewer than V vertices, a cycle exists.

• MST vs shortest path tree: MST minimises total selected tree-edge weight; shortest path tree minimises distances from one source. They are not the same.

• Euler vs Hamilton: Euler uses every edge exactly once; Hamilton visits every vertex exactly once.

• Low-link values: bridge condition is low[child] > tin[parent]; articulation condition for non-root is low[child] >= tin[parent].

• Indexing: confirm whether platform input is 0-indexed or 1-indexed.

• Recursion depth: Java recursion may overflow on a long chain; use iterative traversal when constraints are large.

Part 10 — High-value interview practice ladder

Foundation

BFS traversal; DFS traversal; number of connected components; number of provinces; flood fill; number of islands.

Core patterns

Undirected cycle; directed cycle; bipartite graph; course schedule; course schedule II; shortest path in binary matrix.

Shortest paths

Network delay time; cheapest flights within K stops; path with minimum effort; minimum multiplications; word ladder.

MST / DSU

Connecting cities with minimum cost; Kruskal MST; accounts merge; redundant connection; number of operations to connect network.

Advanced structure

Strongly connected components; critical connections; articulation points; clone graph; alien dictionary.

Flow / matching

Maximum bipartite matching; max flow; min cut; assignment problem.

Tree extension

Tree diameter; LCA; subtree queries; path sum; HLD basics.

Part 11 — What is intentionally beyond the core interview syllabus?

No single handbook can literally contain every graph algorithm used in research, operations research, computational geometry, or competitive programming. The following are valuable specialist extensions after the core set above:

- Minimum-cost maximum-flow and circulation with demands.
- Hopcroft–Karp implementation details and weighted bipartite matching (Hungarian algorithm).
- SPFA (understand its worst-case behaviour; do not treat it as a guaranteed faster Bellman–Ford).
- Offline dynamic connectivity with rollback DSU; fully dynamic connectivity.
- Planarity testing, graph minors, treewidth, and parameterised graph algorithms.
- Advanced shortest paths: contraction hierarchies, bidirectional Dijkstra, replacement paths.
- Advanced tree structures: link-cut trees, Euler Tour Trees, rerooting DP, virtual trees.
- Approximation/randomised algorithms for hard graph optimisation problems.

Interview priority: master BFS/DFS, cycle detection, bipartite, topological sort, shortest paths, DSU, MST, SCC, bridges/articulation points, and tree LCA before spending time on specialist algorithms.

Final preparation checklist

- I can explain the algorithm without code in 60–90 seconds.
- I can state its preconditions and prove why it works at a high level.
- I know time and space complexity and can justify them.
- I can code it in Java without copying.
- I can dry-run it on disconnected graphs, cycles, isolated vertices, and boundary cases.
- I know when not to use it and which algorithm to choose instead.

Note: This handbook is designed as a practical interview reference. Always test implementations against the exact problem constraints, input format, graph direction, edge multiplicity, and weight limits.